Chapter 2

Programming Fundamentals

1. Structure of a Java Program

Java is case-sensitive (e.g., MyClass and myclass are different identifiers).

```
public class HelloWorld {
   public static void main(String[] args) {
      System.out.println("Hello, World!");
   }
}
```

- public class HelloWorld – every Java program needs at least one class, and the class name must match the filename (HelloWorld.java).
- public static void main(String[] args) – the main method is the entry point of every program.
- System.out.println(...) – prints text to the screen.

2. Access Modifiers

Access modifiers control the visibility of classes, methods, and variables. The three commonly introduced are:

- Public – accessible from any class or method.
- Private – accessible only within the class it is declared in.
- Protected – accessible within the same package and by subclasses.

3. Java Syntax Rules

- Every statement ends with a semicolon (;).
- Curly braces { } define a block of code.
- Class names start with an uppercase letter; method and variable names start with a lowercase letter.
- Java is case-sensitive: Hello and hello are different.
- The file name must match the public class name.

4. Comments in Java

Comments explain code to the programmer and are ignored by the compiler.

- // – single-line comment.
- /* ... */ – multi-line comment, used for longer explanations.

5. Data Types

A data type defines what kind of value a variable stores and how much memory it uses. Data types fall into two categories:

5.1 Primitive Data Types

Built-in types that store actual values directly in stack memory. There are 8 primitive types:

Type	Size	Example	Description
byte	1 byte	100	Small integer (-128 to 127)
short	2 bytes	20000	Medium integer (-32,768 to 32,767)
int	4 bytes	50000	Standard integer (most used)
long	8 bytes	123456789L	Large integer
float	4 bytes	5.75f	Decimal (single precision)
double	8 bytes	19.99	Decimal (double precision)
char	2 bytes	'A'	Single character (Unicode)
boolean	1 bit	true / false	Logical value

5.2 Non-Primitive (Reference) Data Types

Created by the programmer; they don't store values directly, but store a reference (memory address) pointing to where the data lives in heap memory — like a house address pointing to where the actual data resides.

Type	Description	Example
String	Sequence of characters (text)	String name = "Alice";
Array	Collection of similar data types	int[] marks = {90, 85, 88};
Class	Blueprint for creating objects	class Student { ... }
Object	Instance of a class	Student s1 = new Student();
Interface	Contract a class must follow	interface Shape { ... }

5.3 Primitive vs Non-Primitive — Key Differences

Feature	Primitive	Non-Primitive
Definition	Basic built-in types	Derived / user-defined types
Stores	Actual values	Reference (memory address)
Memory size	Fixed per type	Variable
Default value	0 / false / \u0000	null
Methods	No built-in methods	Have methods (e.g. length(), toUpperCase())
Mutability	Cannot be changed (value overwritten)	Can be mutable or immutable
Stored in	Stack memory	Heap memory

6. Variables

A variable is a named container in memory that stores a value — like a labeled box holding data.

Syntax: dataType variableName = value;

Example: int age = 20;

6.1 Naming Rules

- Must begin with a letter, $, or _ (e.g., age, $price) — cannot start with a digit.
- Cannot contain spaces.
- Java is case-sensitive: age and Age are different variables.
- Cannot be a reserved keyword (class, int, static, etc.).
- Can contain letters, digits, underscores, and dollar signs.

6.2 Types of Variables

Type	Description
Local Variable	Declared inside a method; used only there.
Instance Variable	Belongs to an object; declared inside a class but outside any method.
Static Variable	Belongs to the class itself, shared by all objects; declared with the static keyword.

7. Type Casting

Type casting converts a value from one data type to another.

7.1 Widening (Automatic / Implicit) Casting

- Converts a smaller data type to a larger one (byte -> short -> int -> long -> float -> double).
- Done automatically by Java, with no data loss.

Example: int num = 10; double d = num; → d becomes 10.0

7.2 Narrowing (Manual / Explicit) Casting

- Converts a larger data type to a smaller one (double -> float -> long -> int -> short -> byte).
- Done manually using parentheses, e.g. (int) d — may cause data or precision loss.

Example: double d = 9.78; int num = (int) d; → num becomes 9 (decimal part truncated).

Characters can also be cast, since they are stored internally as Unicode numbers, e.g. int ascii = 'A'; gives 65, and (char) 66 gives 'B'.

8. Operators in Java

An operator is a symbol that performs an operation on variables or values. Java has 7 main categories:

Type	Purpose / Example
Arithmetic	+, -, *, /, % (e.g., 10 % 3 = 1)
Assignment	=, +=, -=, *=, /= (e.g., a += 5)
Relational (Comparison)	==, !=, >, <, >= (return true/false)
Logical	&&, ||, ! (combine boolean conditions)
Unary	++, -- (increment/decrement, pre and post)
Bitwise	&, |, ^, ~, <<, >> (operate on bits)
Ternary	condition ? valueIfTrue : valueIfFalse

9. Input and Output in Java

9.1 Output

System.out.println("Hello Students!"); and System.out.print("Welcome"); print text/values, with println adding a new line.

9.2 Input (Scanner class)

The Scanner class reads user input. new allocates memory for the Scanner object.

```
import java.util.Scanner;
Scanner sc = new Scanner(System.in);
String name = sc.nextLine();
int age = sc.nextInt();
```

Method	Description
nextBoolean()	Reads a boolean value
nextByte()	Reads a byte value
nextDouble()	Reads a double value
nextFloat()	Reads a float value
nextInt()	Reads an int value
nextLine()	Reads a String value
nextLong()	Reads a long value
nextShort()	Reads a short value